Electrónica Digital 2: Laboratorio 3
Lógica secuencial y máquinas de estados finitos

Isaac Martirena Héctor Pereira
Equipo UTEC City
Ingeniería en Mecatrónica, Universidad Tecnológica (UTEC)
Fray Bentos, Uruguay
isaac.martirena@estudiantes.utec.edu.uy
hector.pereira@estudiantes.utec.edu.uy

***Resumen*—Se diseñaron y validaron dos sistemas síncronos en VHDL sobre una FPGA Spartan-7. En la Parte A se generó un pulso por segundo a partir de 100 MHz y se habilitó un contador de diez bits. En la Parte B se reutilizó el divisor para secuenciar AND, OR, XOR y suma mediante una FSM de Moore y un datapath separado. Se ejecutaron testbenches autoverificables, se completaron síntesis e implementación y se programó la placa por JTAG. Se verificaron la cuenta, el reset, la secuencia y la suma con acarreo. Las restricciones internas de 100 MHz fueron satisfechas; la comprobación física fue visual y no constituyó una medición instrumental de precisión temporal.**

***Index Terms*—FPGA, VHDL, lógica secuencial, divisor de frecuencia, contador, máquina de estados finitos, Moore, datapath, Vivado.**

I. Introducción

En el Laboratorio 3 de Electrónica Digital 2 se abordó el diseño de circuitos síncronos mediante VHDL sobre una FPGA Spartan-7, conforme a la consigna de la asignatura [1]. Se plantearon dos desarrollos relacionados: en la Parte A, un divisor de frecuencia y un contador binario de diez bits; en la Parte B, una máquina de estados finitos (FSM) encargada de secuenciar operaciones lógicas y aritméticas. La habilitación temporal diseñada para la primera parte se reutilizó en la segunda.

La validación fue realizada mediante simulación funcional con aserciones y pruebas visuales en placa. Fueron completadas las síntesis, la implementación y la programación de ambas partes. Los resultados observados fueron contrastados con los modelos de referencia.

I-A. Objetivos

Se establecieron los siguientes objetivos:

- Describir procesos síncronos en VHDL mediante `rising_edge` y reset.
- Calcular e implementar un divisor basado en un contador módulo `MAX`.
- Implementar un contador ascendente con habilitación *clock enable*.
- Describir una FSM de Moore mediante estados enumerados y tres procesos.
- Integrar la FSM como unidad de control y un *datapath* combinacional.
- Contrastar las formas de onda simuladas con el comportamiento de la FPGA.

II. Marco teórico

II-A. Lógica combinacional y secuencial

En un circuito combinacional, las salidas dependen únicamente de las entradas actuales. En uno secuencial, interviene además el estado interno, que conserva información de los ciclos anteriores [4]. En VHDL, un registro se describe mediante `rising_edge(clk)`; el reset síncrono se evalúa dentro de ese flanco, a diferencia del reset asíncrono, que puede actuar sin esperar al reloj [5].

II-B. División de frecuencia y habilitación

Un divisor por módulo se obtuvo mediante una cuenta interna y la detección del valor terminal (*terminal count*) [3]. Para la práctica se especificó un pulso `en` de un ciclo de duración cada `MAX` ciclos del reloj principal. El número de ciclos requerido se calculó como

$$\mathrm{MAX} = \frac{f_{\mathrm{clk}}}{f_{\mathrm{pulso}}}. \tag{1}$$

De este modo, los módulos conservaron el dominio de reloj de 100 MHz y utilizaron `en` para determinar cuándo actualizar su estado. El uso de un único dominio de reloj evitó utilizar una salida lógica como reloj de otro bloque, práctica desaconsejada en FPGA por las dificultades de ruteo y análisis temporal [3]. Este pulso de habilitación no fue una señal cuadrada de 1 Hz: su período fue de un segundo y su duración en alto fue de un único ciclo de 10 ns.

II-C. FSM de Moore y datapath

Una FSM está formada por un conjunto finito de estados, reglas de transición y salidas [4]. En una máquina de Moore, las salidas dependen del estado registrado; en una de Mealy, pueden depender del estado y de las entradas actuales. La enseñanza del curso propuso un patrón de tres procesos: registro de estado síncrono, próximo estado combinacional y salida combinacional [4]. En la Parte B se especificó la codificación Moore `op` correspondiente a cuatro estados; el *datapath*, de naturaleza combinacional, recibió A, B y `op`. Por tanto, la salida `op` depende solo del estado, mientras que el resultado de cálculo también depende de A y B.

III. Metodología

III-A. Materiales y herramientas

Fueron utilizados la Spartan Edge Accelerator Board con dispositivo XC7S15FTGB196-1, el shield UTEC de doce entradas y diez LEDs, alimentación USB y un programador JTAG HS2. El desarrollo, la simulación XSim y la implementación fueron realizados con Vivado 2025.1.

III-B. Parte A: divisor de frecuencia

Se diseñó la interfaz del módulo `divisor` con entradas `clk` y `rst`, salida `en` y genérico `MAX`. Se especificó un contador interno que se reiniciaba al alcanzar $\mathrm{MAX} - 1$ y activaba `en` durante un único ciclo. Para un reloj de 100 MHz y un pulso por segundo, se obtuvo

$$\mathrm{MAX}_{1\,\mathrm{Hz}} = \frac{100 \times 10^6}{1} = 100\,000\,000. \tag{2}$$

Para obtener 1 kHz, el valor correspondiente fue $\mathrm{MAX}_{1\,\mathrm{kHz}} = 100\,000$. El contador interno del divisor de 1 Hz requirió $\lceil \log_2(100\,000\,000) \rceil = 27$ bits. El valor terminal se estableció en $\mathrm{MAX} - 1$, no en MAX, para evitar un ciclo adicional.

III-C. Parte A: contador e integración

Se definió un contador ascendente con genérico `N=10`, reset síncrono y habilitación `en`. Su salida `q(9 downto 0)` se asignó a los diez LEDs, con `LED0` como bit menos significativo. En el módulo superior, la salida `en` del divisor se conectó a la entrada homónima del contador. Así, el registro solamente incrementaba cuando recibía un pulso de habilitación. El rango de diez bits fue de 0 a 1023, seguido de desborde a cero.

La Fig. 1 muestra las conexiones del top de la Parte A.

III-D. Comportamiento temporal esperado

El pulso `en` fue especificado como una habilitación de un único ciclo de reloj cada `MAX` ciclos. Con `MAX=4`, la separación entre dos activaciones consecutivas fue de cuatro períodos de reloj. Con `MAX=100 000 000` y reloj de 100 MHz, el intervalo fue de un segundo. En un contador binario, cada bit dividió por dos la frecuencia de la señal cuadrada representada por el bit inmediato inferior [3]. El avance de la cuenta permaneció sujeto a `en`, no a un flanco de una señal generada por lógica combinacional.

III-E. Parte B: FSM y diagrama de estados

Se definieron los estados enumerados `S_AND`, `S_OR`, `S_XOR` y `S_ADD`. Cada pulso de habilitación provocó, por especificación, una transición al siguiente estado y, después de `S_ADD`, el retorno a `S_AND`. El reset se definió con destino `S_AND`. Se separaron el registro de estado, la lógica combinacional del próximo estado y la decodificación de la salida `op`, siguiendo el patrón de tres procesos presentado en el material del curso [4]. La habilitación se consideró dentro de la lógica de transición o del registro, sin generar un segundo reloj.

Tabla I
Codificación y salidas de la Parte B

op	Estado	Cálculo	Salidas
00	AND	$A \wedge B$	LED3:0
01	OR	$A \vee B$	LED3:0
10	XOR	$A \oplus B$	LED3:0
11	ADD	$A + B$	LED4:0

III-F. Parte B: datapath e integración

El datapath recibió `A(3:0)`, `B(3:0)` y `op(1:0)`. Se definieron las operaciones y la visualización indicadas en la Tabla I. La suma se representó en cinco bits para conservar el acarreo, mientras que las operaciones lógicas se expresaron en cuatro bits. El divisor de la Parte A fue incorporado como fuente de habilitación para la FSM.

III-G. Simulación y testbenches

Para la Parte A se planteó un escenario de prueba con `MAX=4`, que permitió observar los pulsos sin simular cien millones de ciclos entre habilitaciones. Se fijaron como criterios de verificación la duración de un ciclo de `en`, su periodicidad, el incremento unitario, el desborde y el reset. En el material de clase se recomendó medir períodos entre eventos con `now` y utilizar `assert`; al comparar valores después de un flanco, se propuso un retardo breve para permitir la actualización de los ciclos delta del simulador [3]. Para la Parte B se establecieron dos recorridos completos de la secuencia, un reset desde un estado distinto de `S_AND` y casos de prueba para cada operación, incluida una suma con acarreo. El material de FSM mostró la comparación de salidas esperadas mediante `assert`, práctica aplicable a la secuencia de `op` [4]. La periodicidad de en, la cuenta, el desborde y el reset fueron comprobados mediante assert en la Parte A; la secuencia, la retención, el reset y todos los pares de operandos por operación fueron comprobados mediante assert en la Parte B. Las capturas fueron utilizadas como evidencia visual complementaria.

III-H. Restricciones e implementación en la placa

La consigna estableció el reloj de 100 MHz en el pin H4 y requirió la restricción temporal `create_clock`. Las entradas y salidas del shield utilizaron el estándar `LVCMOS33`. Además, la salida `AR_3V3_EN` quedó definida como activa en bajo, con asignación al pin L13 y valor lógico cero para habilitar la alimentación de los dip-switches [1]. Se cotejaron las conexiones con el esquema del shield y el XDC maestro de la SEA. El reset se asignó a D2 (A10), posición 4 del DIP chico. Los operandos se agruparon en el DIP grande: A[0:3] en posiciones 5,6,7,8 y B[0:3] en posiciones 1,2,3,4. En la Parte B, A[0:3] se conectó a D9,D8,D7,D6 (A2, B3, A3, A4) y B[0:3] a D13,D12,D11,D10 (H2, H1, B1, B2). Los diez LEDs físicos, LED1 a LED10, se conectaron a LED[0] a LED[9], respectivamente, mediante FPGAIO9 a FPGAIO0. Se completaron síntesis, colocación y ruteo de ambas partes en Vivado 2025.1 para el dispositivo XC7S15FTGB196-1. Se

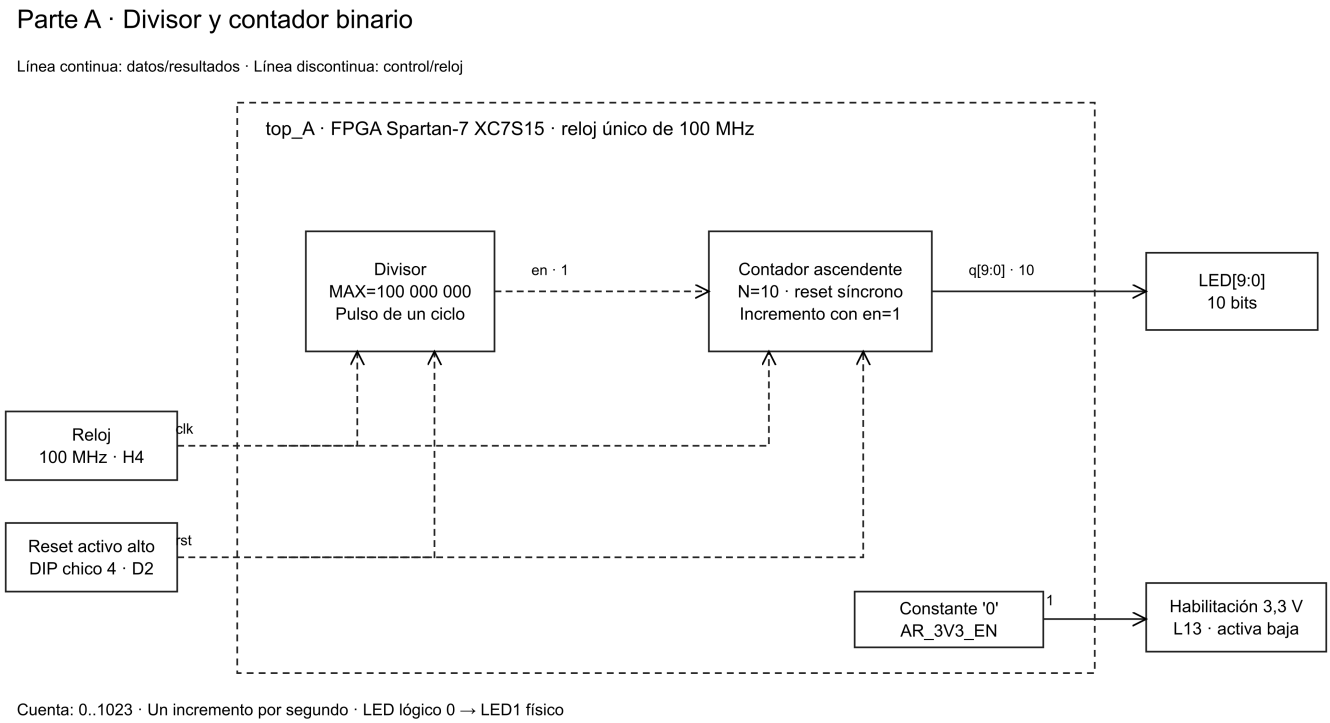


Figura 1. Integración de la Parte A: divisor y contador bajo un único reloj, reset común y salida de diez bits hacia los LEDs.

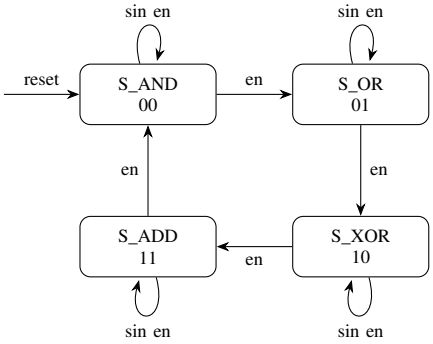


Figura 2. Diagrama de estados de la FSM implementada. El reset tuvo prioridad sobre la habilitación y estableció S_AND desde cualquier estado.

Tabla II
ASIGNACIÓN FINAL DE LOS DIP-SWITCHES

Señal	DIP/posición	Arduino	FPGA
rst	Chico 4	D2	A10
A0	Grande 5	D9	A2
A1	Grande 6	D8	B3
A2	Grande 7	D7	A3
A3 (MSB)	Grande 8	D6	A4
B0	Grande 1	D13	H2
B1	Grande 2	D12	H1
B2	Grande 3	D11	B1
B3 (MSB)	Grande 4	D10	B2

programaron ambas partes por JTAG. La validación física y su registro se documentaron en la sección de resultados.

La Tabla II resume la asignación física final utilizada en las pruebas.

IV. RESULTADOS Y DISCUSIÓN

IV-A. Parte A: simulación

Fueron implementados y analizados los módulos `divisor`, `contador` y `top_A`, el archivo de restricciones y el testbench `tb_parte_A`. Los cuatro archivos VHDL se analizaron con Vivado 2025.1 y se ejecutó el testbench con XSim. La simulación utilizó un reloj de 10 ns, `MAX=4` y un contador de tres bits para observar el desborde en un intervalo breve.

Durante 41 ciclos se compararon mediante `assert` el pulso de habilitación y la cuenta con modelos de referencia. Se consumieron diez habilitaciones y se observó el desborde de siete a cero. Antes de activar el reset final se comprobó explícitamente que la cuenta era dos; permaneció en dos hasta el siguiente flanco ascendente y volvió a cero en dicho flanco. Todas las aserciones finalizaron correctamente a los 455,001 ns. El contador consume el enable registrado en el flanco anterior.

La Fig. 4 muestra la ejecución corregida. La prueba ejercitó divisor y contador con N=3, mientras que el top de diez bits fue contrastado por separado en placa.

IV-B. Parte A: funcionamiento físico

El bitstream de la Parte A fue cargado por JTAG y fue confirmado el estado DONE. En la placa fue observado el avance

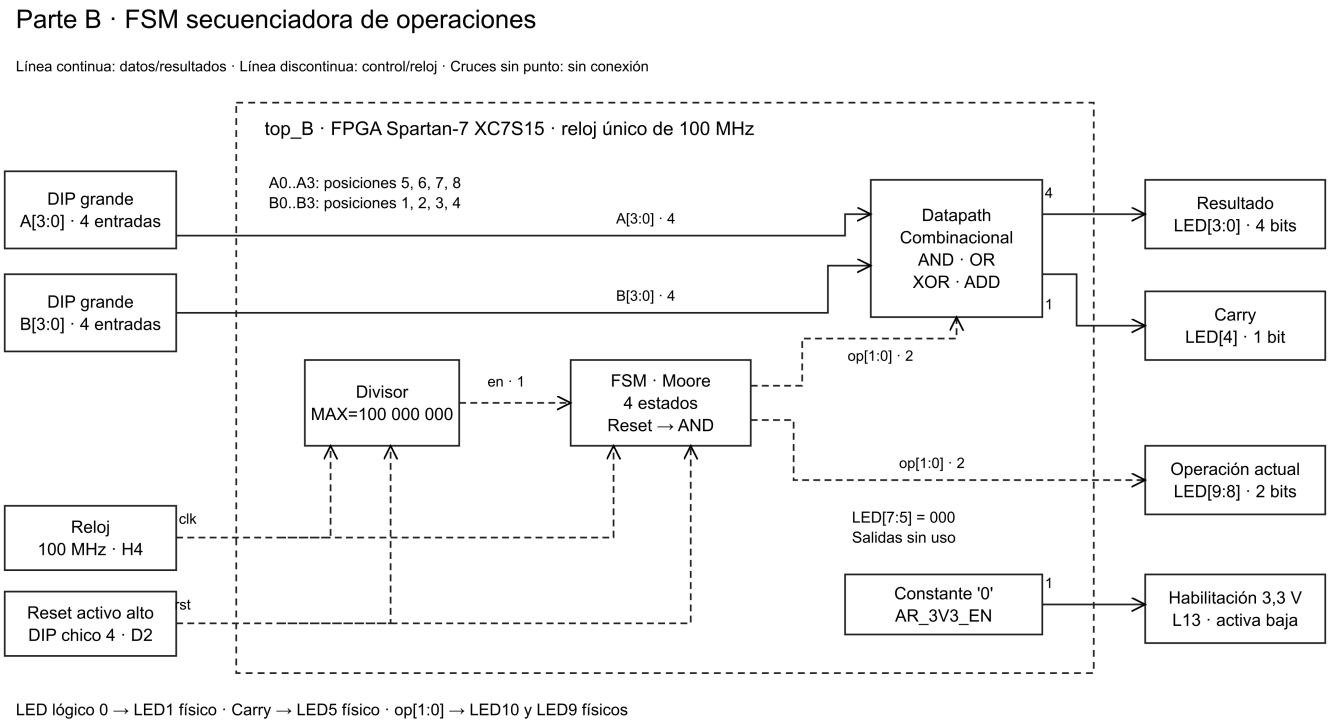


Figura 3. Integración de la Parte B: entradas A y B, divisor, FSM, datapath y salidas. El reloj y el reset fueron comunes al divisor y a la FSM; las salidas LED[7:5] fueron fijadas a cero.

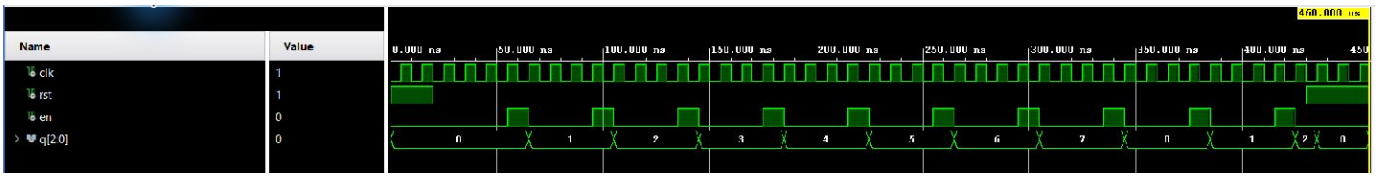


Figura 4. Simulación corregida de la Parte A: reloj de 10 ns, MAX=4 y N=3. Se observa el desborde de siete a cero y el reset síncrono final que borra la cuenta desde dos.

binario de los diez LEDs a una velocidad visual compatible con un incremento por segundo. El LED1 físico, asociado al bit lógico LED[0], alternó su nivel con cada incremento. Al mantenerse activo el reset del DIP chico, posición 4, la cuenta fue puesta a cero; al desactivarlo se reanudó el avance. Fue conservada una grabación de esta prueba. La duración exacta del período no fue medida con instrumental externo y el desborde físico completo de 1024 estados no fue ensayado; el desborde fue verificado en simulación.

IV-C. Parte B: simulación

Se implementó una FSM enumerada de tres procesos y un datapath combinacional, integrados con el divisor de la Parte A. El testbench del top utilizó MAX=4 y reloj de 10 ns, recorrió más de tres ciclos de estados y verificó la retención entre habilitaciones. Se ensayaron los 256 pares posibles de operandos para cada operación, incluida la suma con acarreo, y se comprobó que los operandos no alteraban la salida de Moore. El reset se aplicó desde OR, verificando su acción síncrona, el retorno a AND y el reinicio de la temporización. Todas las aserciones finalizaron correctamente a los 605,001 ns en XSim 2025.1. La asignación física de las entradas A y B se completó según el esquema; las operaciones fueron contrastadas posteriormente en placa. La Fig. 5 muestra la secuencia de operación y el reset desde OR. Los cambios rápidos de operandos al comienzo corresponden al barrido exhaustivo del testbench; la comprobación de esos casos se realizó mediante aserciones y requiere una ampliación temporal para su inspección visual.

IV-D. Parte B: comprobación visible del acarreo

Se ejecutó una prueba adicional con operandos constantes A=15 y B=3 durante dos vueltas completas. En ADD se comprobó mediante aserciones el resultado de cinco bits 10010 (18 decimal), carry=1 y los cuatro bits inferiores 0010. En AND, OR y XOR se obtuvieron 3, 15 y 12, respectivamente, con carry=0. La ejecución finalizó sin fallos de aserción a

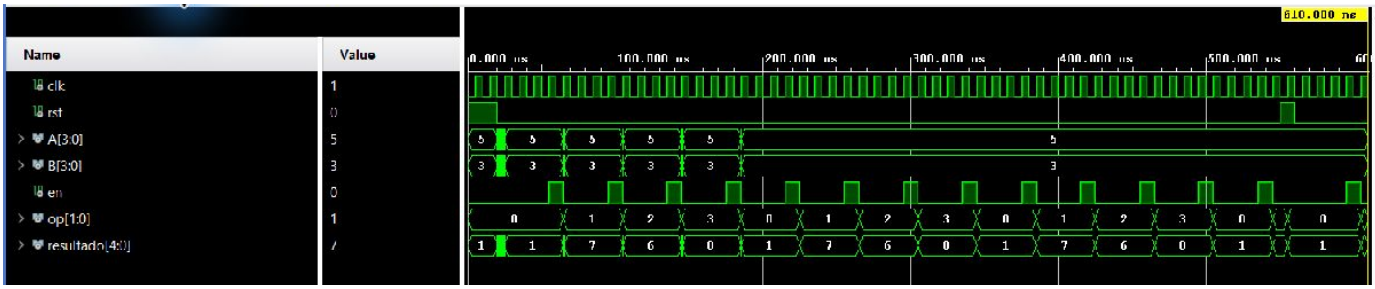


Figura 5. Simulación integrada de la Parte B con MAX=4: reloj, reset, operandos, habilitación, código de operación y resultado de cinco bits. op=0,1,2,3 corresponde a AND, OR, XOR y ADD.

Tabla III
REGISTRO DE PRUEBAS DE LA FSM Y DEL DATAPATH

A	B	op	Esperado	Observado
0101	0011	00	00001	00001
0101	0011	01	00111	00111
0101	0011	10	00110	00110
0101	0011	11	01000	01000
1111	0011	11	10010	10010

los 415,001 ns. La Fig. 6 muestra estos resultados y el reset posterior desde OR.

IV-E. Parte B: validación física

Con A=5 y B=3 fueron observados los resultados 1, 7, 6 y 8 al recorrerse AND, OR, XOR y ADD, respectivamente. En los LEDs físicos se obtuvieron LED1; LED1, LED2 y LED3; LED2 y LED3; y LED4. El código de operación fue mostrado en LED10 y LED9 físicos. Con A=15 y B=3 fue verificada la suma 18 mediante LED5 físico (acarreo) y LED2 físico (valor dos en los cuatro bits inferiores). El reset fue comprobado desde un estado diferente de AND: se retornó a op=00 y al resultado AND de los operandos, sin borrar las entradas. Tras la primera prueba, la asignación de switches fue reorganizada para reunir A y B en el DIP grande y colocar sus MSB en las posiciones 8 y 4. La numeración final fue utilizada en las pruebas posteriores. Fueron conservadas dos tomas de funcionamiento de la Parte B.

La Tabla III compara los valores calculados con los patrones observados visualmente en placa para los casos ensayados. Los cinco bits de salida se representaron como `LED4:0`.

IV-F. Discusión y limitaciones

No fueron reportadas discrepancias funcionales en los casos físicos ensayados. En la simulación inicial de A se detectó que el reset final ocurría cuando la cuenta ya era cero, por lo que no demostraba su borrado. El testbench fue extendido y fue exigida una cuenta previa de dos: así se comprobó el retorno síncrono a cero. En B, el barrido exhaustivo de operandos resultó demasiado rápido para ser interpretado en una captura general; fue añadida una prueba con operandos constantes para exponer el acarreo. La dificultad de lectura física de entradas fue resuelta agrupando los operandos en el DIP grande y documentando explícitamente su orden. Estas correcciones mejoraron la cobertura de las pruebas y su interpretación sin modificar las operaciones solicitadas.

La comprobación exhaustiva de operandos fue realizada en simulación; la placa fue probada con casos representativos. No fue caracterizada la metastabilidad o el rebote de los switches. Las tomas de placa fueron conservadas como registro visual; la evidencia completa de entrega también requirió las grabaciones de las simulaciones.

IV-G. Síntesis e implementación

Ambos diseños completaron síntesis, colocación y ruteo con reloj de 100 MHz. En la Parte A se utilizaron 15 LUT y 38 registros, con margen mínimo de setup de 5,897 ns y de hold de 0,251 ns. En la Parte B se utilizaron 16 LUT y 32 registros, con márgenes de 5,913 ns y 0,251 ns, respectivamente. No se registraron violaciones de setup ni hold en las rutas restringidas.

Los informes temporales advierten que las entradas de switches y las salidas hacia LEDs no tienen retardos externos especificados. Por tanto, estos resultados acreditan las restricciones internas del reloj, no una caracterización temporal completa de las interfaces externas ni el funcionamiento físico del shield. La comprobación en placa fue visual, con operandos estáticos.

El DRC final no registró violaciones tras incorporar las propiedades de configuración indicadas en el XDC maestro. Se generaron los bitstreams de ambas partes; ambos fueron programados por JTAG y ensayados visualmente en la placa.

V. PREGUNTAS DEL LABORATORIO

V-A. Parte A

V-A1. Circuito combinacional, secuencial y proceso síncrono: La distinción principal fue la presencia de memoria: un circuito combinacional entregó una función de las entradas presentes y uno secuencial conservó un estado entre ciclos. En VHDL, `rising_edge(clk)` delimitó el instante de actualización de los registros. Con reset síncrono, su acción quedó sujeta al mismo flanco, lo que hizo posible un comportamiento temporal determinista.

V-A2. MAX para 1 Hz y 1 kHz y anchura del contador: Aplicando la Ec. (1), se obtuvo $\mathrm{MAX}_{1\,\mathrm{Hz}} = 100\,000\,000$ y

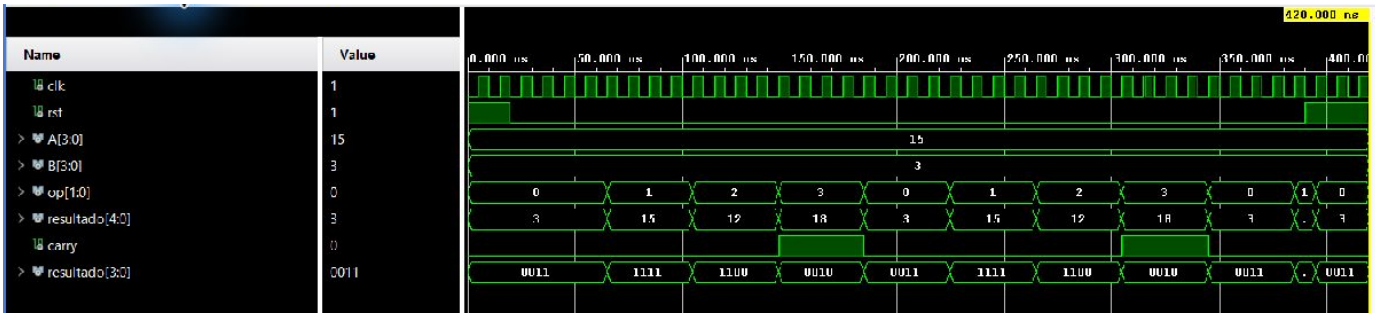


Figura 6. Prueba de acarreo con A=15 y B=3. En op=3 (ADD), resultado[4:0]=18, carry=1 y resultado[3:0]=0010.

$\mathrm{MAX}_{1\,\mathrm{kHz}} = 100\,000$. Para representar los valores comprendidos entre cero y 99 999 999, fueron suficientes 27 bits porque $2^{26} = 67\,108\,864 < 100\,000\,000 \leq 2^{27} = 134\,217\,728$.

V-A3. Habilitación en lugar de reloj dividido: El empleo de `clock enable` mantuvo el contador y el divisor sincronizados con el mismo reloj físico. Así se evitó tratar una señal generada por lógica ordinaria como una nueva red de reloj, lo que habría complicado la distribución, las restricciones temporales y el análisis de cruces entre dominios. La operación de un registro quedó condicionada a `en='1'` sin alterar su fuente de reloj.

V-A4. Uso de genéricos en simulación: Con el valor real de cien millones de ciclos por pulso, una simulación de varios incrementos implicaría una cantidad muy elevada de eventos, tal como se explicó en el caso de estudio del material de divisores [3]. El parámetro genérico `MAX` permitió sustituir el valor de implementación por uno pequeño, como cuatro, sin modificar la estructura lógica. En el testbench de A fueron utilizados MAX=4 y N=3, y las comprobaciones finalizaron a los 455,001 ns.

V-A5. Frecuencia de conmutación de los LEDs: Para incrementos a 1 Hz, el bit i del contador cambió de nivel cada 2^i segundos, y completó un período de encendido y apagado cada 2^{i+1} segundos. En consecuencia, su frecuencia de señal cuadrada fue

$$f_{\mathrm{LED}i} = \frac{1\,\mathrm{Hz}}{2^{i+1}}. \tag{3}$$

Por ejemplo, `LED0` tuvo una frecuencia de señal de 0,5 Hz (una transición por segundo), `LED1` de 0,25 Hz y `LED9` de $1/1024$ Hz. Esta distinción evitó confundir la frecuencia de incremento del contador con la frecuencia de parpadeo de cada LED.

V-B. Parte B

V-B1. Definición y componentes de la FSM: La FSM se representó mediante un registro de estado, una lógica de transición y una función de salida. Los cuatro estados de la Fig. 2 representaron, en orden, AND, OR, XOR y ADD. Cada habilitación especificó el paso siguiente; después de ADD se regresó a AND. El reset estableció el estado inicial AND.

V-B2. Diferencia entre Moore y Mealy: En una máquina de Moore, la salida codificada depende exclusivamente del estado almacenado; en una máquina de Mealy, también puede depender de las entradas actuales. La FSM propuesta fue de Moore porque `op` se obtuvo del estado. Aunque los resultados del datapath variaron combinacionalmente al modificarse A o B, ello no convirtió a la FSM de control en una máquina de Mealy.

V-B3. Estados enumerados y síntesis: La descripción mediante un tipo enumerado permitió utilizar nombres semánticos y redujo errores asociados a códigos binarios asignados manualmente. En general, la síntesis transforma los estados simbólicos a una representación física apropiada y puede escoger una codificación distinta de la codificación de salida `op`, según sus opciones de optimización. Por tanto, la numeración de `op` no implicó necesariamente que el registro físico de estado utilizara exactamente esos dos bits.

V-B4. Separación de procesos síncronos y combinacionales: El registro de estado se planteó en un proceso síncrono para conservar el estado entre flancos y actualizarlo solamente al recibirse una habilitación. La lógica de próximo estado se planteó combinacionalmente para evaluar las reglas de transición en función del estado actual. La salida de Moore se definió como función combinacional de ese estado. En el registro de la FSM fue utilizada la condición `elsif en='1' then estado <= siguiente;` dentro de `rising_edge(clk)`, con prioridad del reset. En procesos `process(all)` separados fueron definidas las transiciones mediante `case estado` y la decodificación de `op`. De ese modo no se crearon registros adicionales en la lógica combinacional.

V-B5. Relación con la arquitectura de un procesador: La FSM cumplió una función de control al seleccionar la operación mediante `op`. El datapath realizó las transformaciones de datos sobre A y B y generó los resultados. Esta separación fue análoga, en una escala sencilla, a la organización de un procesador: la unidad de control determina la acción y la ruta de datos la ejecuta. La arquitectura del laboratorio no constituyó un procesador completo, ya que no incorporó un programa de instrucciones ni todos sus registros y rutas de acceso.

VI. Conclusiones

Fue verificada la reutilización de una habilitación periódica para controlar dos comportamientos secuenciales distintos: la cuenta binaria y el avance de una FSM. La secuencia

observada en placa fue consistente con las simulaciones, mientras que el reloj de 100 MHz fue mantenido como único dominio en ambos diseños. La división temporal fue lograda mediante clock enable, sin utilizar una salida lógica como reloj adicional.

La separación entre control y datapath fue validada al mantenerse el código de operación independiente de los operandos. Para A=5 y B=3 fueron obtenidos los resultados 1, 7, 6 y 8; en la suma 15+3 fue preservado el quinto bit, con lo que fue evitada la pérdida del acarreo. El reset de la FSM fue distinguido del borrado de datos: se regresó a AND, pero los operandos permanecieron definidos por los switches.

La cobertura de prueba fue mejorada al detectarse que un reset aplicado sobre una cuenta ya nula no demostraba el comportamiento buscado. También fue comprobado que una verificación exhaustiva podía ser correcta y, a la vez, poco clara visualmente; por ello fue añadida una prueba estable de acarreo. La reorganización de los DIP redujo la ambigüedad de lectura de entradas. Como mejora posterior fue identificada la incorporación de sincronización de switches y un registro sistemático de las pruebas físicas. Los márgenes temporales internos positivos y el DRC sin violaciones respaldaron la implementación, aunque no sustituyeron la comprobación del hardware ni una medición instrumental del período.

Evidencias y archivos

Los archivos de entrega fueron organizados en fuentes de la Parte A (divisor, contador y top), fuentes de la Parte B (FSM, datapath y top), testbenches y restricciones XDC. Las capturas de simulación fueron incorporadas en las Figs. 4–6. Fueron conservadas una toma de placa de la Parte A y dos de la Parte B. Al cierre de esta versión, la edición de los dos videos finales con sus segmentos de simulación y la incorporación de sus enlaces de entrega permanecieron pendientes.

Referencias

[1] Universidad Tecnológica (UTEC), “Laboratorio 3: Lógica secuencial y máquinas de estados finitos, Partes A y B,” Electrónica Digital 2, guía de laboratorio, 2026.

[2] Universidad Tecnológica (UTEC), “Rúbrica de evaluación por niveles de logro: Laboratorio 3,” Electrónica Digital 2, 2026.

[3] Universidad Tecnológica (UTEC), *Todo divisor de frecuencia es, en el fondo, un contador*, material didáctico de Electrónica Digital 2, s. f.

[4] Universidad Tecnológica (UTEC), *Sistemas que se comportan distinto según su historia*, material didáctico de Electrónica Digital 2, s. f.

[5] AMD, *Vivado Design Suite User Guide: Synthesis (UG901)*. [En línea]. Disponible: https://docs.amd.com/r/en-US/ug901-vivado-synthesis

[6] AMD, *Vivado Design Suite User Guide: Logic Simulation (UG900)*. [En línea]. Disponible: https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation

[7] AMD, *Vivado Design Suite User Guide: Using Constraints (UG903)*. [En línea]. Disponible: https://docs.amd.com/r/en-US/ug903-vivado-using-constraints